Report of ML Challenge, Data Analytics Laboratory (DA5401)

Harshil Pandey, DA24C006

November 2024

Contents

1 Introduction

The final model used for submission was One-vs.-Rest Logistic Regression. For Data Engineering, Principal Component analysis was used. For evaluating the model, data was randomly split into Train and Test subsets with 80% of data in Train. Sparse representations were used to save memory and tackle memory errors.

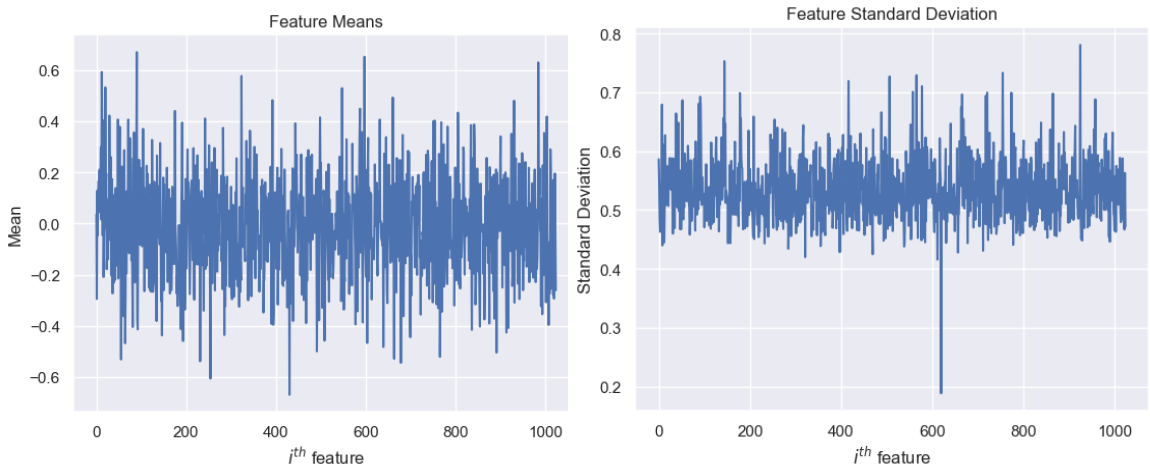
2 Exploratory Data Analysis

As part of Exploratory Data Analysis(EDA), I first checked if there were any null values present in the dataset or not. There were no null values in any of the samples. Further, I checked the mean, standard deviation, minimum, and maximum values of each feature, which suggested that the scale of the features were similar. *Figure 1* and *Figure 2* summarize the findings. We can see that the mean, standard deviation, minimum, and maximum values of all features are not significantly different. The standard deviation of the feature means was 0.198 which further suggests that all features have similar range and scale.

After exploring the features, I explored the labels as well. I calculated the correlation between each label and visualized it with a heat map. Analyzing the correlation of labels allows us to discover relationships between the labels, for example, if the correlation between two labels is high, then there is a high chance

that if one label is present, then the other is also present. If the correlation is highly negative then, it suggests that if one label is present, the other is not. *Figure 3* to *Figure 6* give the most significantly correlated labels.

Figure 7 and *Figure 8* plot the labels with the most and least frequencies respectively. We can observe that there are labels that only have 1 sample in the whole dataset, it will be very difficult to predict these labels.



(a) Feature Means (b) Feature Standard Deviations

Figure 1: Descriptive Statistics(a)

(a) Minimum Values (b) Maximum Values

Figure 2: Descriptive Statistics(b)

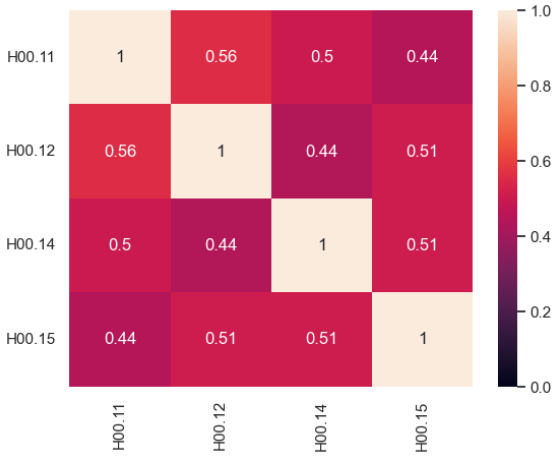


Figure 3: Correlation Heat Map 1

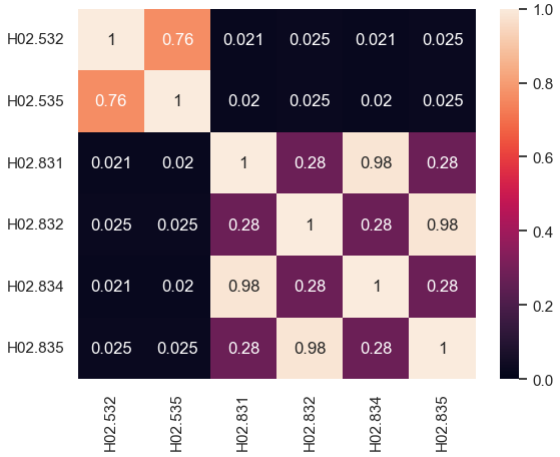


Figure 4: Correlation Heat Map 2

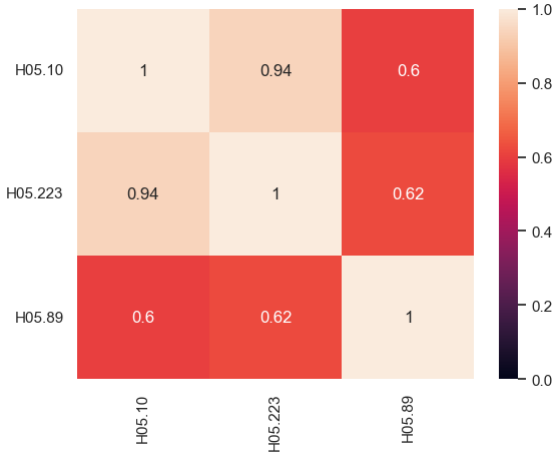


Figure 5: Correlation Heat Map 3

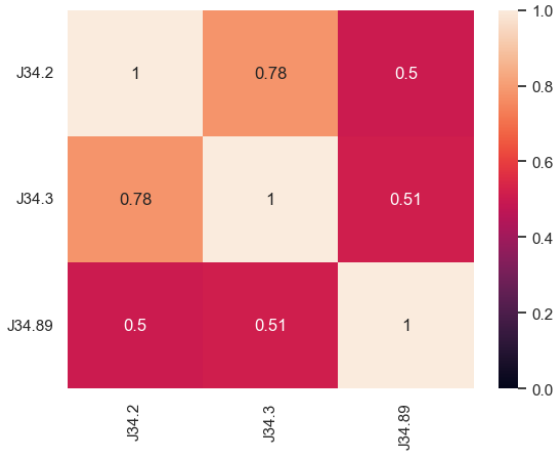


Figure 6: Correlation Heat Map 4

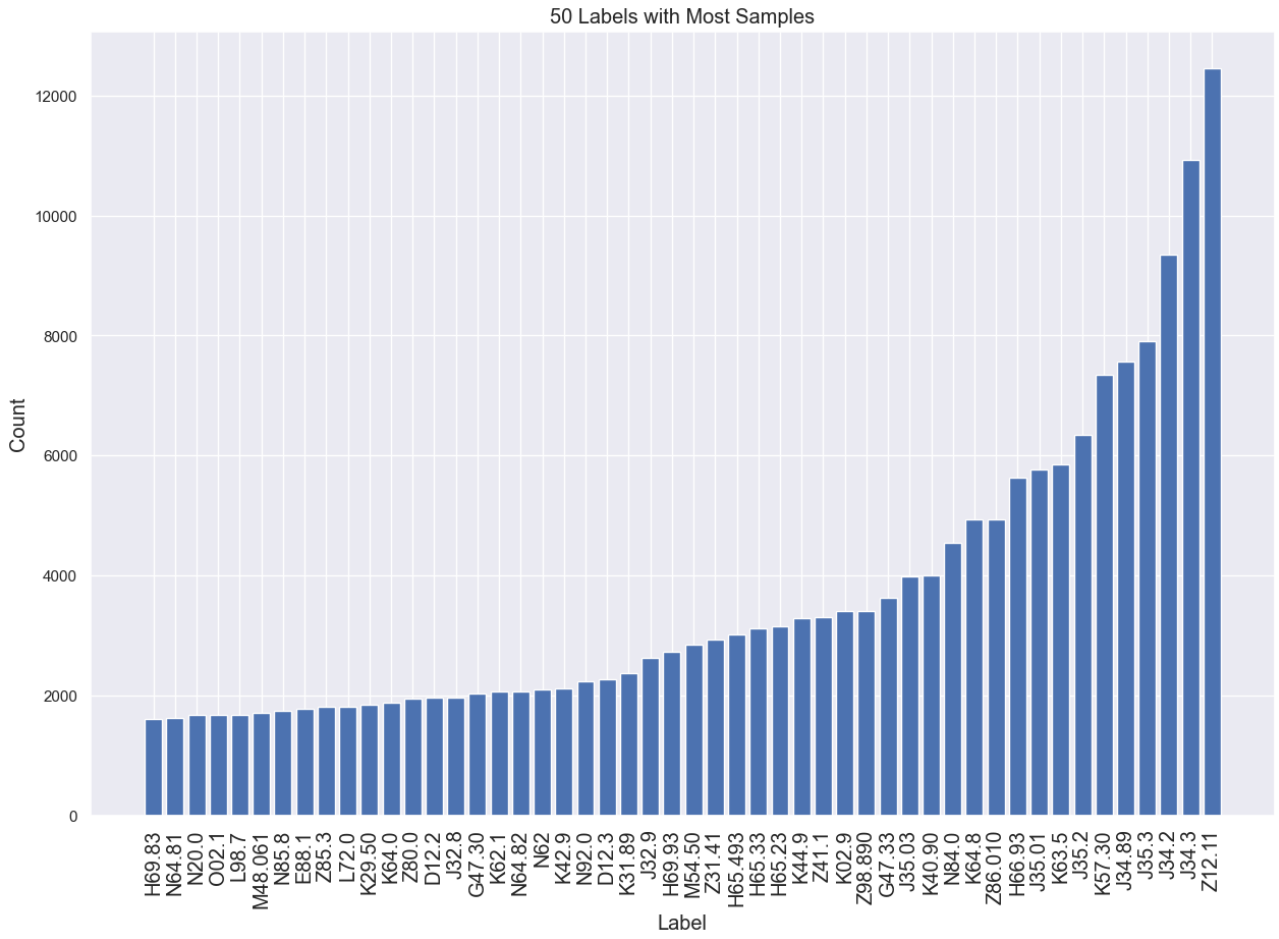


Figure 7: Labels with the most frequency

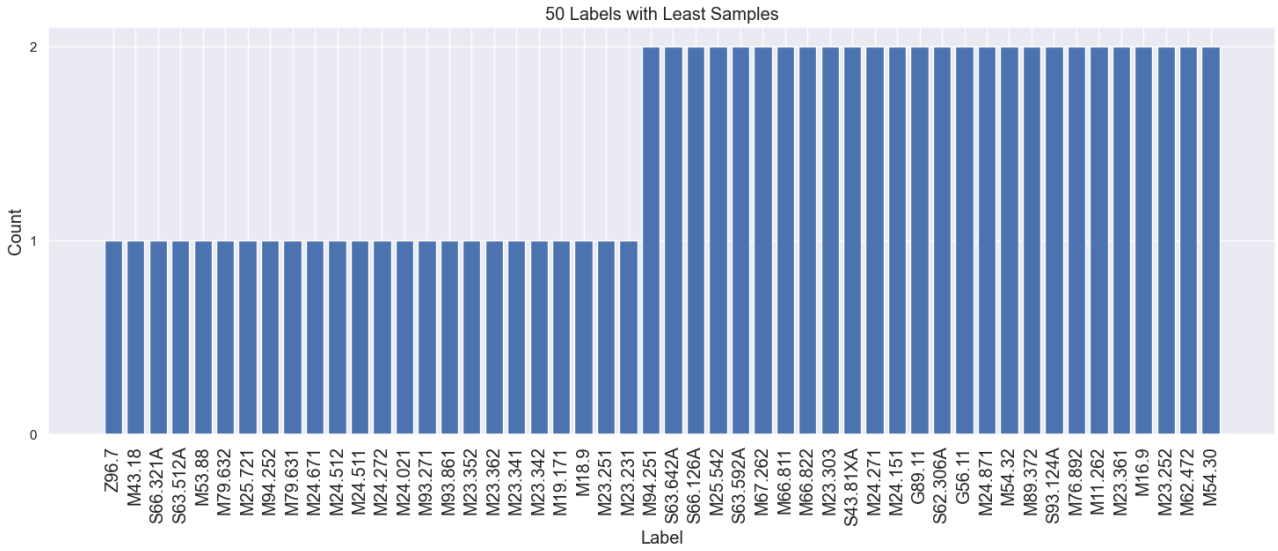


Figure 8: Labels with the least frequency

3 Data Engineering

Since the scale and range of the features were similar, I did not scale the data. To reduce the dimensionality of the dataset, I used Principal Component Analysis(PCA). *Figure 9* visualizes the results of the PCA algorithm. We can observe that 95% of the variance is obtained using the top 466 principal components out of the 1024 present. Using the obtained features reduces the computational time required by our algorithm. Moreover, the obtained features are de-correlated which reduces the effect of high dimensionality.

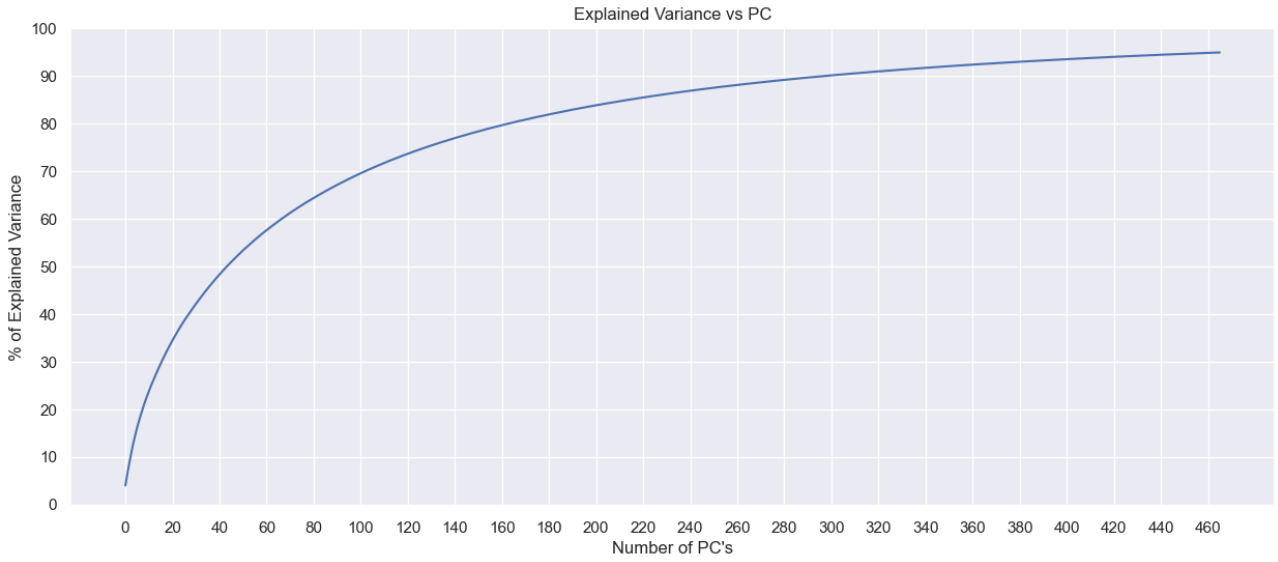


Figure 9: Explained Variance by Principal Components

4 Details of the Model

The model is very simple, each label has a separate classifier trained on the training data, and the corresponding labels are used. These individual classifiers are `scikit-learn.linear_model.LogisticRegression` objects. Each classifier is stored in a list for future predictions. For predicting a sample, each classifier is asked for a prediction and they are combined in an array to return as the final output. The model is implemented in the `MultiLogReg` class in the code and follows a similar API of `scikit-learn` estimators.

5 Reducing the Memory Usage

Since there are 1400 classifiers, one for each label, the memory usage is huge. But, I observed that the label matrix is only composed of 0's and 1's. Thus it is appropriate to use a sparse representation for the label matrix. The problem now was to use sparse labels, but sparse labels are not accepted by the `scikit-learn` estimators. For this reason, I created my own class to to this exact job of

accepting sparse labels and using them for training instead of directly using `OneVsRestClassifier` utility of `scikit-learn` library. Without this step, I was getting Memory Allocation Errors. The `scipy` library of Python allowed me to use the Compressed Row Matrix to alleviate the Memory Allocation problems.

6 Performance of the Model

The model was evaluated on the F2-Micro metric. The performance on the training and testing subsets is as follows:

Train Score: 87.39
Test Score: 79.56

After getting this performance I decided to finalize it. To submit the predictions, I trained my model on the whole dataset to use the maximum information, the performance on the whole dataset was 84.31.